

看了很多文章依旧不会写 Skill？保姆级攻略请查收！

原创 TRAE TRAE.ai 2026年5月28日 17:06 北京



本文作者：咸鱼，TRAE 开发者用户

「一切皆可 Skill | SOLO 技能创作赛」正在火热进行中，立即了解：

<https://forum.trae.cn/t/topic/16860>

点击查看：[📖 「一切皆可 Skill」 | SOLO 技能创作赛正式开启！](#)

01

为什么你需要一个 Skill，而不是反复教 Agent？



图 1：把重复解释沉淀成 Skill

如果你最近开始频繁和 Agent 协作，很容易遇到一个微妙的问题：它不是完全不会做，而是每次都像第一次做。你刚刚纠正过的格式、提醒过的风险、跑通过的工具顺序，换个任务又要重新

解释一遍。

这篇文章想解决的就是这个问题：哪些经验值得从一次聊天里拿出来，沉淀成 Agent 下次可以主动调用的工作方法。Skill 不是把模型训练得更懂你，而是把一类任务的做法写成可复用的流程，让 Agent 少猜、少漏、少返工。

一句话： Skill 是给 Agent 使用的可复用工作能力包。它把一类任务的触发条件、操作流程、参考资料、工具脚本和验收标准放在一起，让 Agent 遇到同类问题时可按既定方法做，而不是每次重新猜你的要求。

从重复解释说起

这种重复不是抽象问题，它通常落在很具体的工作细节里：同样是整理文档，今天要重新告诉它排版要求；同样是代码 Review，明天还要提醒它先看 diff、再看测试、最后列风险；同样是调用内部工具，换一次任务就可能把参数顺序写错。

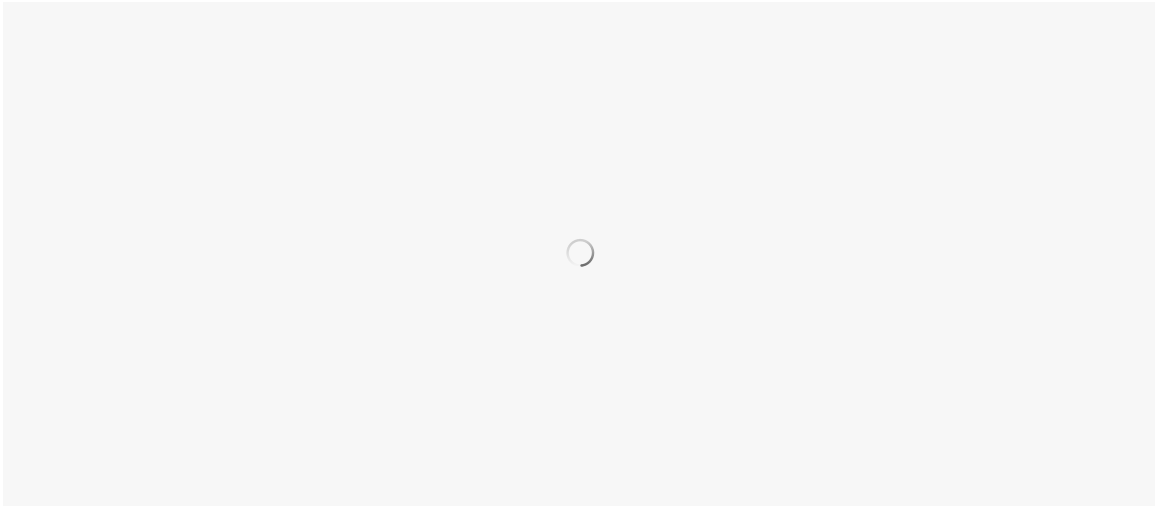
这类问题的本质不是模型“不聪明”，而是任务里的经验没有被沉淀下来。人类同事做熟一件事以后，会形成习惯、检查表和操作路径；Agent 也需要这样的东西——只不过给 Agent 用的不是口头经验，而是结构化的 Skill。

写 Skill 的目的是让 Agent 在同类任务里少猜一点、少漏一步、少重复问一遍背景。你可以把它理解成：把个人或团队反复做过、反复修正过、反复踩坑后总结出来的做法，整理成 Agent 下次可以直接读取和执行的操作手册。

什么是 Skill：可被 Agent 调用的工作能力包

更具体地说，Skill 通常是一个目录。目录里最核心的是 [SKILL.md](#)：它告诉 Agent 这个 Skill 适用于什么任务、开始前要读什么、应该按什么顺序做、哪些情况必须停下来确认。目录里还可以放脚本、模板、参考资料和例子。

所以 Skill 不是“把知识灌进模型里”，也不是让模型永久学会某个技能。它更像一个可被 Agent 打开的工作手册：平时只暴露简短说明，真正触发后才把详细流程读进上下文。



一个 Skill 里面到底放什么？

一个 Skill 最小可以只有 `SKILL.md`，但好用的 Skill 往往会把“触发、执行、资料、验证”分开写清楚。

- `description`：给 Agent 做第一眼匹配。它要说清楚“用户说什么时应该触发这个 Skill”。
- `SKILL.md`：核心说明书。写触发边界、操作步骤、必须遵守的限制、输出格式和失败处理。
- `references/`：长规范、API 文档、业务背景、模板。只有执行中需要时才读取。
- `scripts/`：把稳定、机械、容易出错的动作交给脚本，比如校验、转换、批量生成。
- `examples/`：放好结果、坏结果和边界样例，让 Agent 能对照判断质量。
- `evals/`：放最小评测集。每次改 Skill 后，用它检查有没有把原本会做的事情改坏。

核心原理：渐进式加载到底是什么？

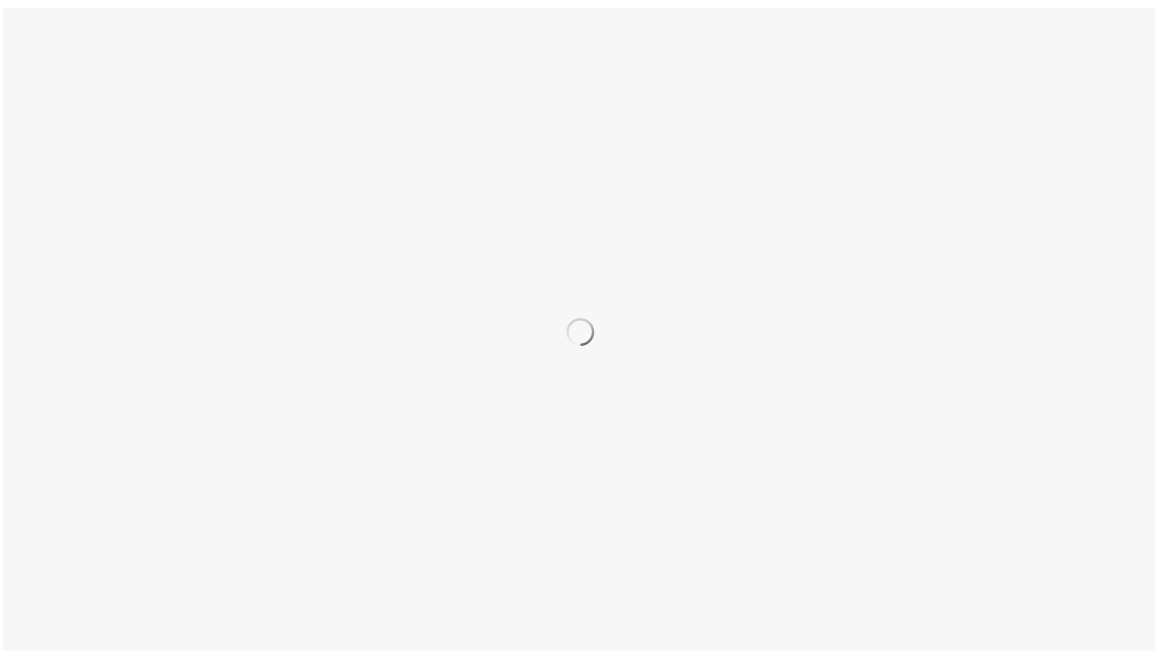
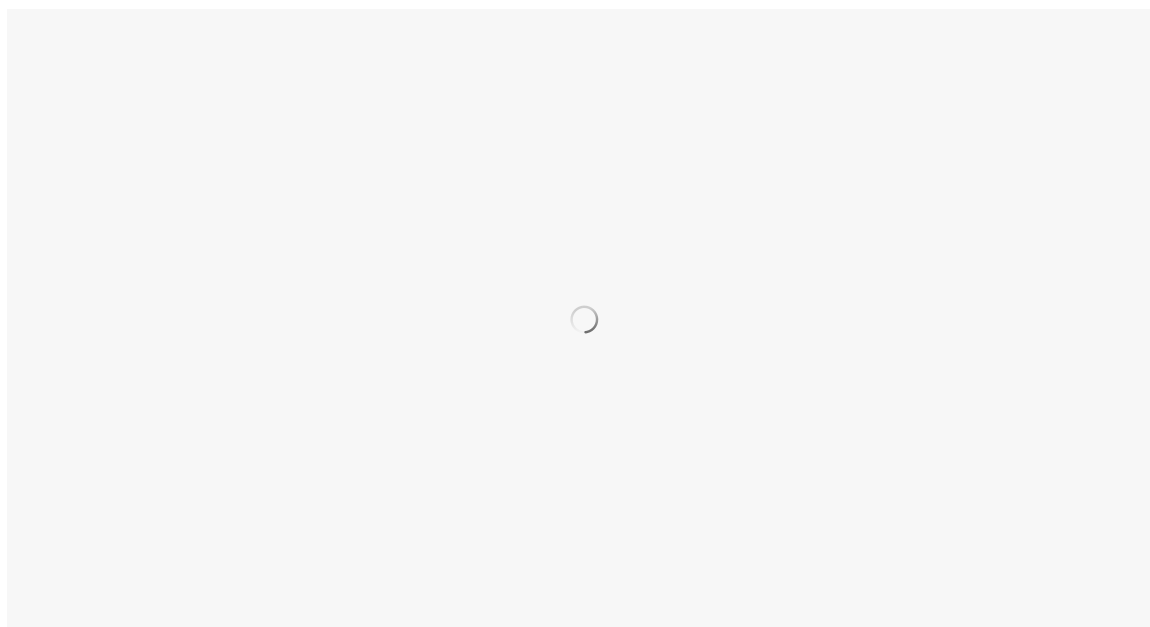


图 2：Skill 的渐进式加载机制

理解 Skill，最关键的一点是：Agent 并不是一开始就把所有 Skill、所有参考资料、所有脚本都读进来。那样上下文会很快被塞满，真正当前任务需要的信息反而被淹没。Skill 采用的是渐进式加载：先用很短的信息判断要不要用，再逐层打开更详细的说明。

第一层：只暴露触发信息

平时 Agent 只需要看到 Skill 的名称和 description。description 的作用不是讲完整教程，而是回答一个问题：用户提出什么任务时，应该考虑使用这个 Skill？



第二层：触发后才读取 SKILL.md

当用户的任务和 description 匹配时，Agent 才进入第二层，打开 [SKILL.md](#)。这一步相当于从“我知道有这个能力”切换到“我知道这件事应该怎么做”。好的 [SKILL.md](#) 不会只写原则，而会写清楚动作顺序：先检查什么、读取什么、调用什么工具、遇到权限或失败时怎么处理、最后按什么标准交付。

第三层：需要时再打开资料和脚本

执行过程中，如果 [SKILL.md](#) 发现任务需要更长的背景，才会让 Agent 去读 [references/](#)；如果任务里有确定的机械动作，才会运行 [scripts/](#)。这就是“渐进”的含义：不是把所有内容一次性放到模型面前，而是在每一步只拿当前必要的信息。

为什么这件事重要？

渐进式加载解决的是上下文管理问题。Agent 的上下文不是无限的，塞进去的内容越多，干扰也越多。如果一个文档 Skill 同时包含写作规范、飞书 API、图片上传、排版规则、失败处理和几十个示例，Agent 每次整理一段文字都读完整套资料，**反而更容易抓不住当前重点。**

所以好的 Skill 会把内容拆成“入口短、主干清楚、资料可追溯、动作可执行”。入口短，才能准确触发；主干清楚，才能稳定执行；资料可追溯，才能在复杂场景里补细节；动作可执行，才能把容易出错的步骤交给脚本或 CLI。

判断一个 Skill 是否写得好，可以先看它有没有做到渐进式加载：description 是否足够准确，[SKILL.md](#) 是否只放核心流程，长资料是否拆到 [references/](#)，稳定动作是否交给 [scripts/](#)。如果所有内容都堆在一个超长文件里，Agent 不是更“懂”，而是更容易被噪声带偏。

强模型写 Skill，弱模型执行流程

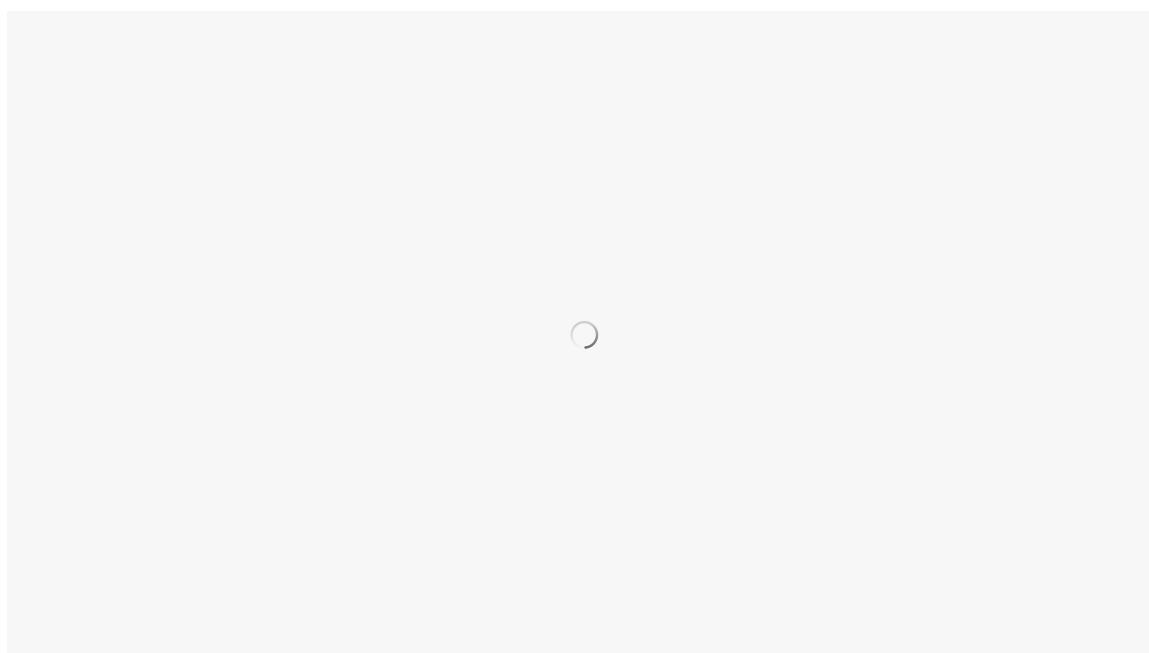
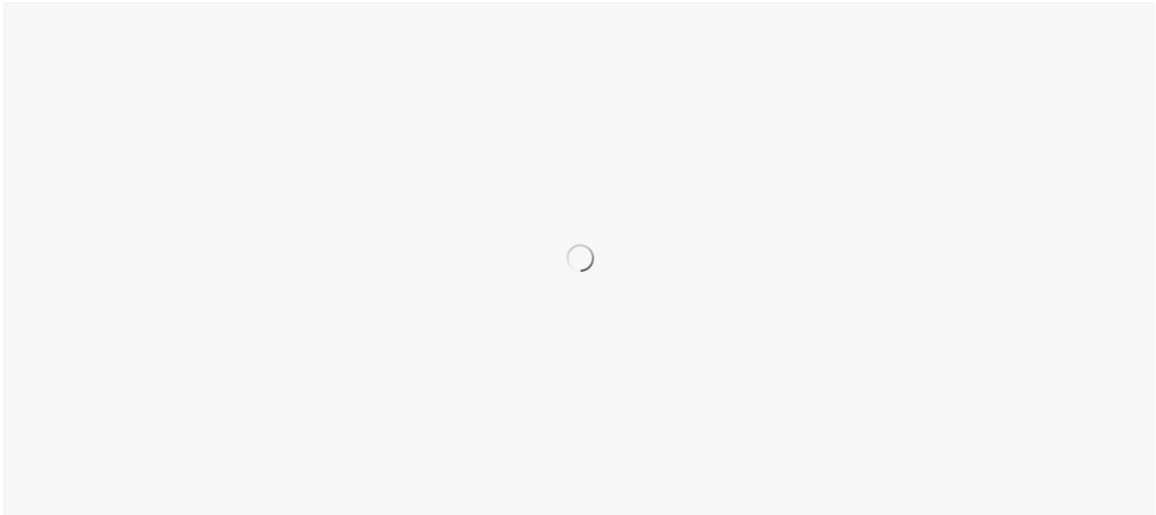


图 3：强模型、弱模型与人的分工

渐进式加载还有一个很实际的价值：它把“写流程”和“跑流程”分开了。复杂、模糊、需要归纳经验的部分，可以交给更强的模型来整理；一旦流程被写成 Skill，后续很多同类任务就可以由更便宜、更快、能力较弱的模型按步骤执行。



所以 Skill 的终点不是“模型自己总结了一套说明”，而是“人把流程约束清楚以后，模型可以稳定照着做”。强模型可以帮你写，弱模型可以帮你跑，但流程的边界、责任和验收标准，最终还是要靠人来定。

Skill 解决的四类核心问题

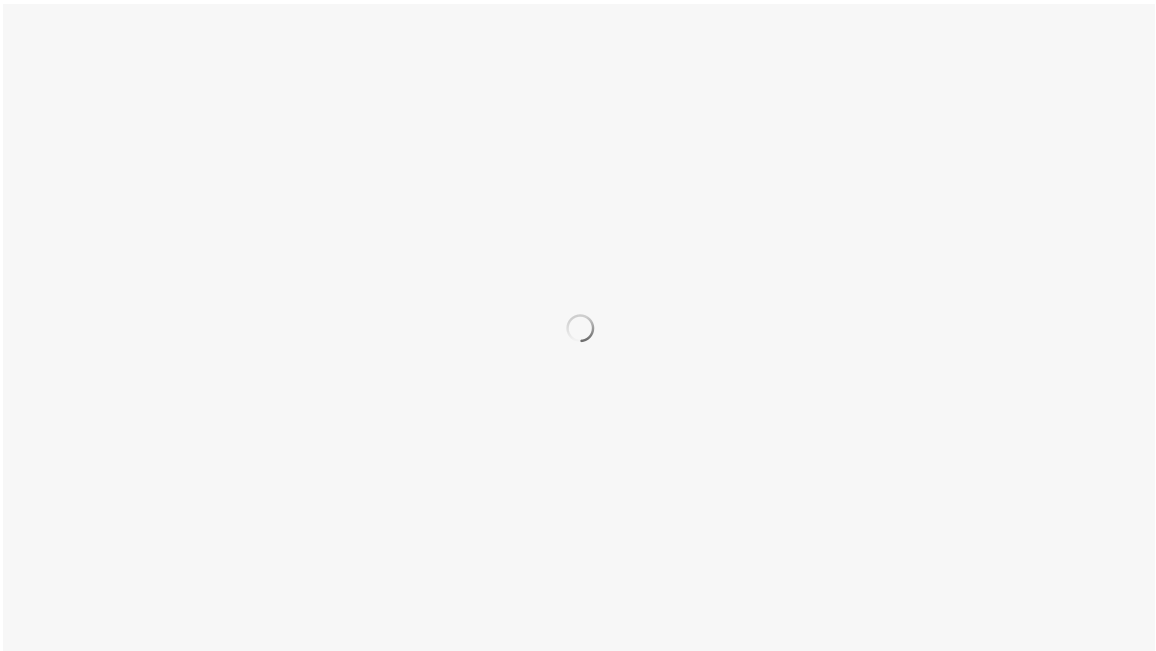
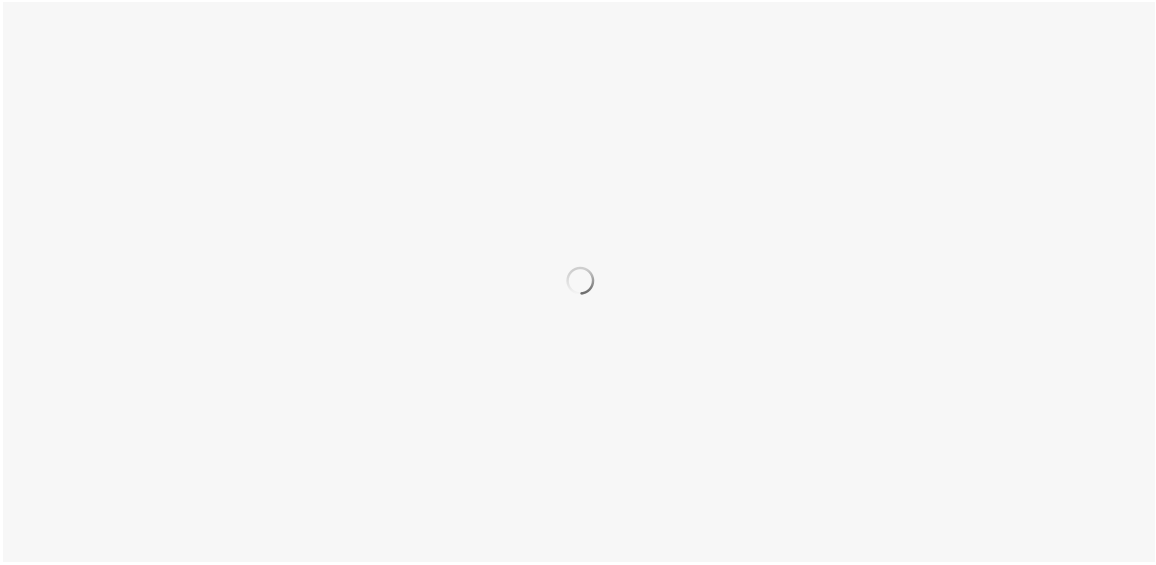
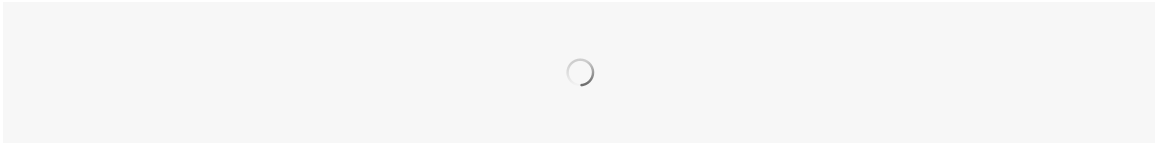


图 4：Skill 解决的四类核心问题



不是所有任务都值得 Skill 化。下一步要先选场景，而不是马上动手写。



什么样的任务值得沉淀成 Skill?

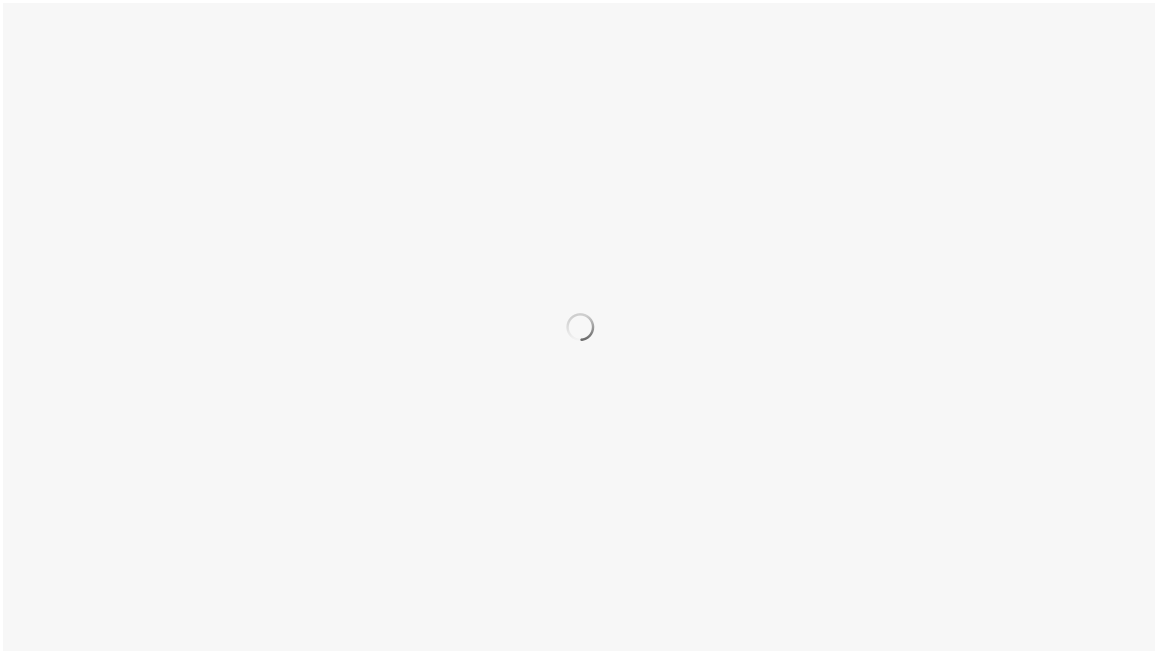


图 5：判断任务是否值得 Skill 化

最容易失败的开局是想写一个"万能助手 Skill"。万能通常意味着边界模糊，边界模糊就意味着 Agent 自由发挥。更稳的起点是挑一个小而真实的场景，它最好满足三个条件：重复出现、容易跑偏、有明确交付物。



先跑 baseline，再决定要不要 Skill 化

一个更稳的判断方法是：先不要写 Skill，拿 3-5 个真实 case 让 Agent 在没有 Skill 的情况下完整跑一遍。你要观察的不是它这次回答得漂不漂亮，而是它是否稳定暴露出同一类问题：总是漏掉某个检查步骤、总是误用同一个工具、总是输出无法验收，或者每次都需要你重新解释同一套口径。



每个 Skill 都是一笔上下文税。 新增一个 Skill，就等于给 Agent 多加一条需要判断的路线。一个边界模糊、很少触发、没人维护的 Skill，不但不会提高效率，反而可能污染路由、浪费上下文，甚至让原本能做对的任务变复杂。

适合作为第一批的场景

- **需求澄清** — 输入经常模糊，但产出格式可以固定（REQUIREMENT.md、问题清单、风险点）。
- **技术方案生成** — 流程相对稳定，需要统一模块、接口、取舍说明（DESIGN.md、接口表、任务拆解）。
- **飞书文档整理** — 有固定阅读体验和排版要求（可发布文档、摘要、待确认事实）。
- **会议纪要加工** — 信息来源固定，输出可标准化（结论、决策、待办、风险）。

- **代码 Review** — 关注点明确，容易形成检查清单（缺陷、风险、测试缺口、建议修改）。

三种常见拆法

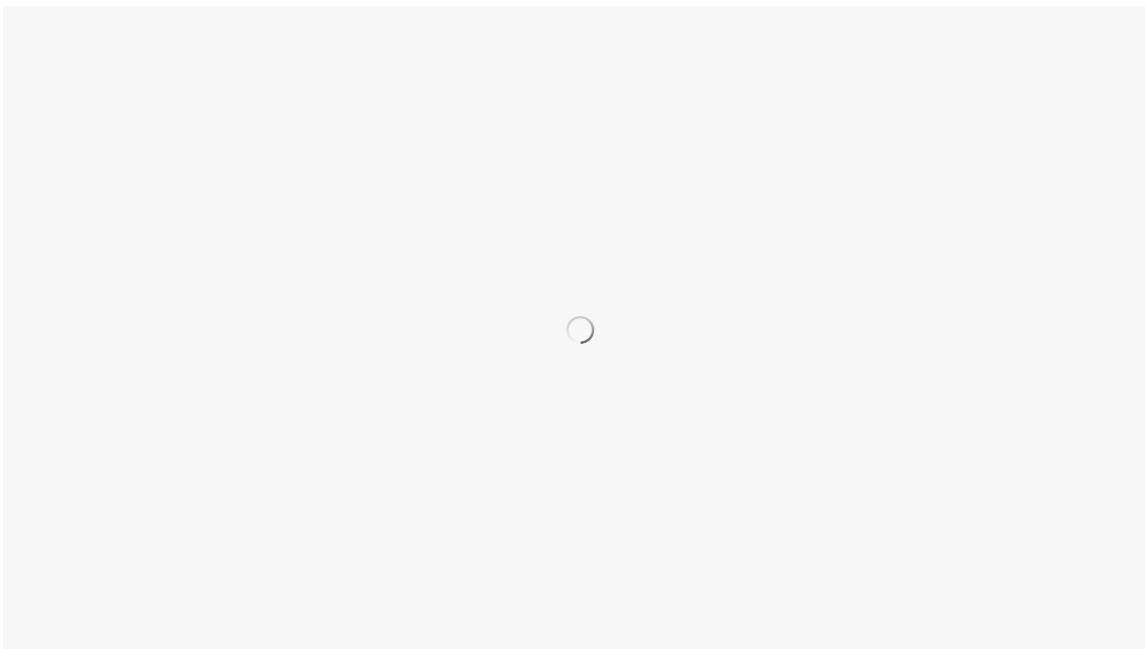
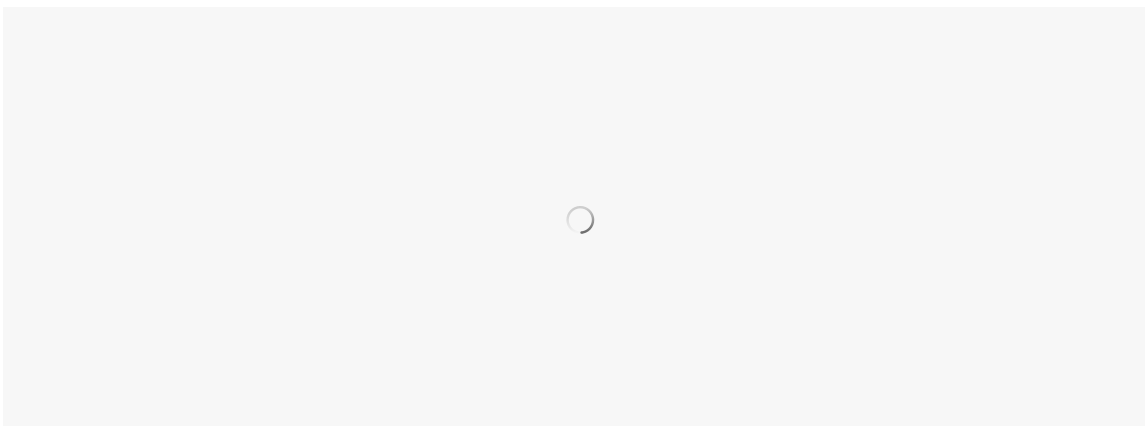


图 6：Skill 的三种常见拆法

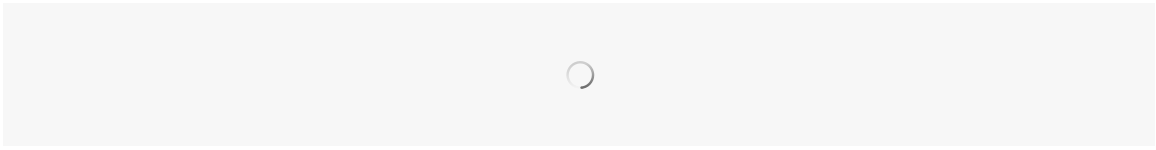
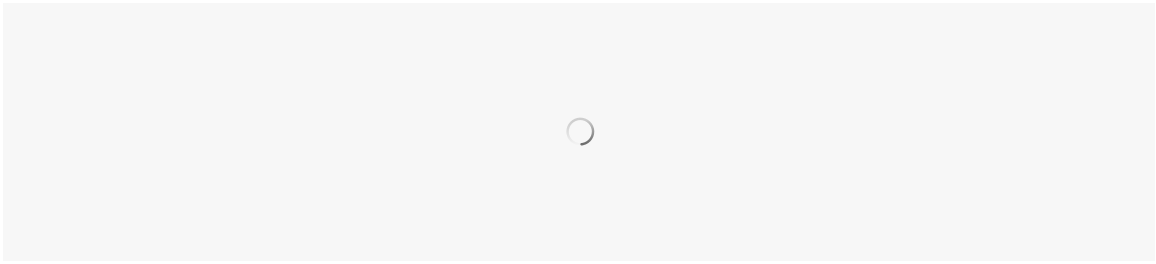


这三种拆法可以组合，但不建议都塞进一个 Skill。边界越清楚，后面越好维护。

贯穿案例：把飞书草稿整理成知识库文章

接下来用一个固定例子往下走：把飞书草稿整理成团队知识库文章。这个例子足够日常，也足够麻烦，因为它同时涉及阅读原文、重组结构、保留来源、标注待确认事实和输出到固定模板。

如果每周都要做一次，你很快会发现：标题风格、读者对象、callout 用法、发布前检查，靠临场提醒很容易漏。后面讲找现成 Skill、改造 Skill、从零写最小版和做评测，都会围绕这个场景展开。



现成 Skill 应该怎么找、怎么选？

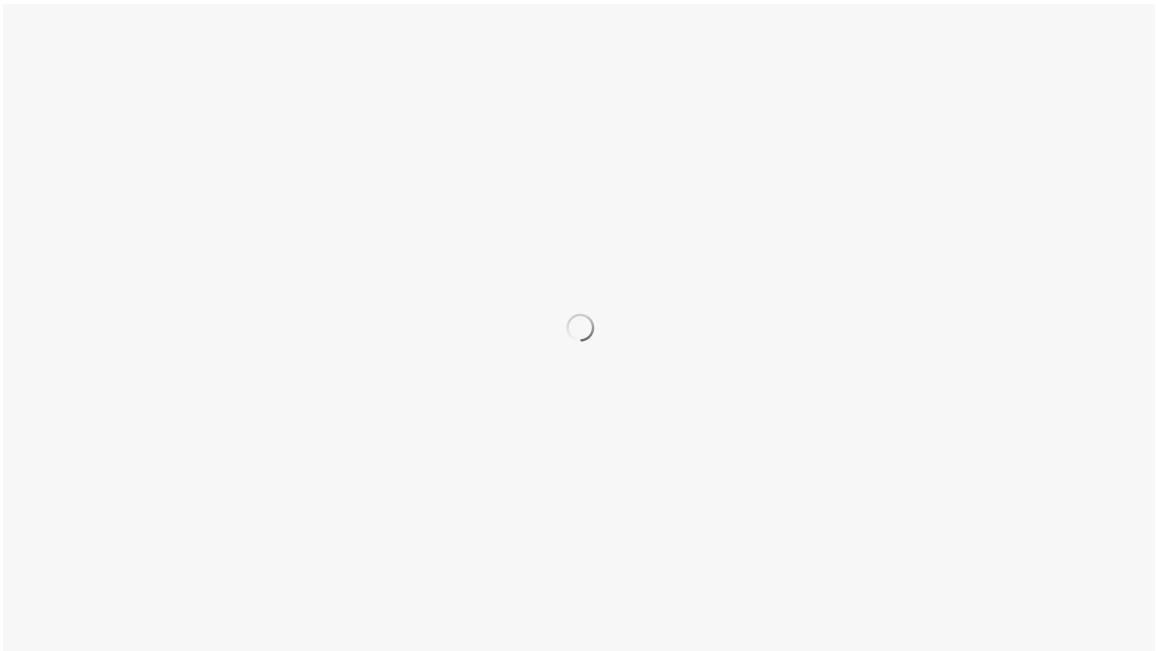


图 7：现成 Skill 的查找、筛选和安装路径

写 Skill 之前，先搜一下有没有现成的。很多常见场景社区已经有人做过，能复用就先复用，能改造就别从空白页开始。

去 **skills.sh** 搜索

skills.sh 是一个开放的 Agent Skills 目录。你可以按关键词搜（react、testing、docs、review、deploy 等任务词），也可以看 All Time、Trending、Hot 榜单快速判断某个领域有没有成熟 Skill。



回到前面的飞书草稿整理案例，可以先搜 [docs](#)、[writing](#)、[knowledge base](#)、[review](#) 这类任务词。不要只看名称是否像，而要看它是否真的覆盖“读取草稿、重组结构、保留来源、标注待确认事实、输出可发布文档”这条链路。

find-skills: 找 Skill 的 Skill

如果你经常需要找 Skill，可以先装 [find-skills](#)。它本质上是一个元 Skill：不直接帮你写代码或文档，而是教 Agent 怎么发现、筛选、推荐和安装其他 Skill。

```
安装 find-skills  
npx skills add https://github.com/vercel-labs/add-skill --skill find-skills
```

用命令行搜索和安装

```
搜索和安装 Skill  
# 按关键词搜索 Skill  
npx skills find react performance  
npx skills find pr review  
npx skills find changelog  
  
# 安装一个 Skill 集合或仓库  
npx skills add vercel-labs/agent-skills  
  
# 按详情页提示安装某个 Skill  
npx skills add https://github.com/vercel-labs/add-skill --skill find-skills
```

不要看到安装量高就直接装。Skill 可能包含脚本、工具调用和文件操作。安装前至少读一遍 [SKILL.md](#) 和 [scripts/](#)，看清它会让 Agent 做什么。

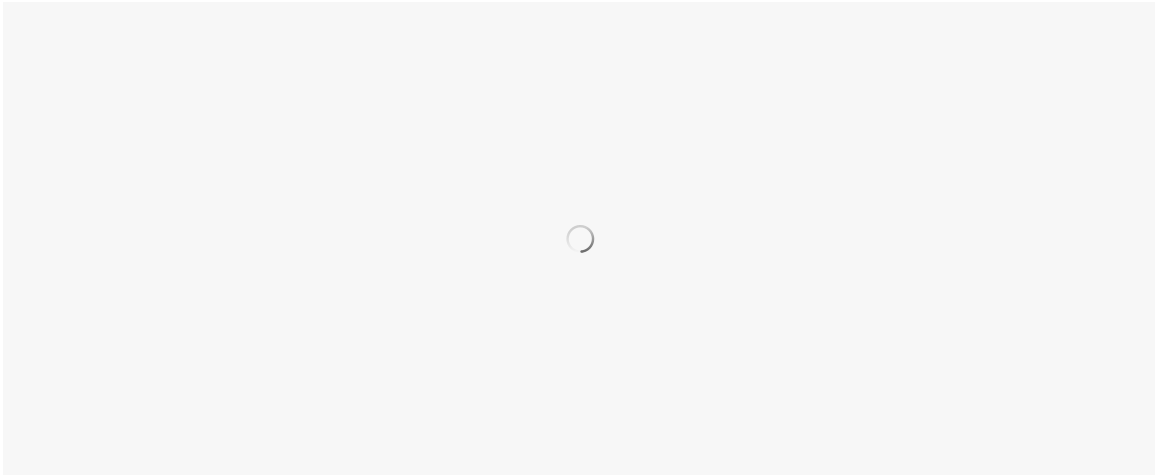
怎么判断一个 Skill 值不值得用



好 Skill 的最小结构

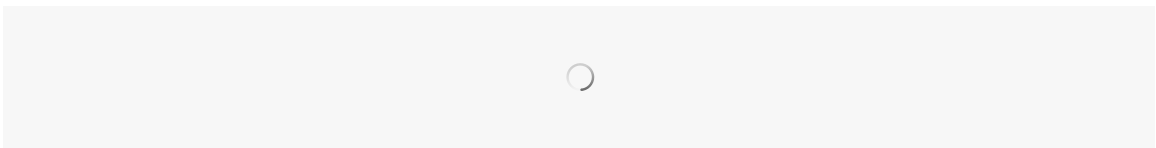
- **元数据** — Agent 什么时候该想起它？描述不能像口号。
- **触发条件** — 哪些请求必须用，哪些不要用？不能只写"当用户需要时使用"。
- **输入规范** — 需要哪些材料？缺材料怎么办？不能让 Agent 自己猜业务背景。
- **工作流程** — 先做什么后做什么？哪里必须校验？不能只有原则没有动作。
- **输出契约** — 交付物长什么样？不能没有格式也没有示例。
- **失败处理** — 什么情况下要停、重试或问人？不能默认 Agent 会自己判断风险。
- **示例和评测** — 怎么知道它做得对？不能只凭一次演示判断可用。

文档助手 description 的好坏对比



直接用、改造后用，还是自己写

- **直接用** — 场景高度匹配，来源可信，说明清楚，脚本行为可接受。用 3-5 个真实任务试跑。
- **改造后用** — 大方向匹配，但你有自己的流程、模板、术语或风格。保留核心结构，改 description、示例、输出格式和停止条件。
- **自己写** — 没有合适 Skill，或任务强依赖内部系统、私有流程、安全边界。用现成 Skill 当参考，从最小场景开始写。



找到 Skill 以后，怎么改成自己的工作流？

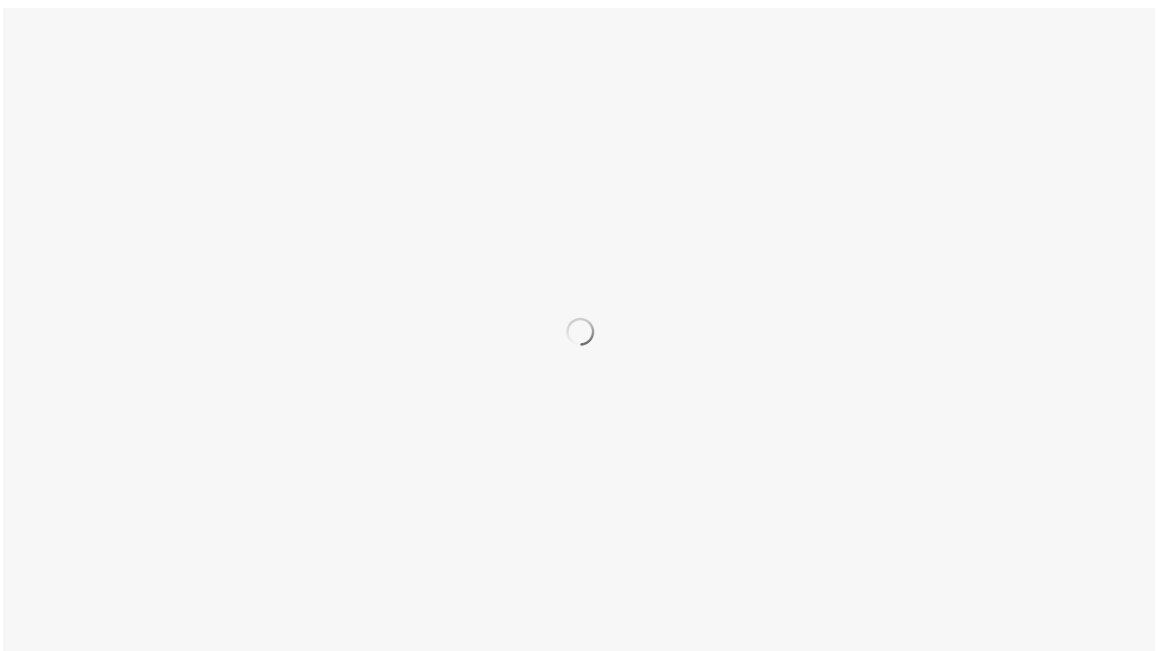


图 8：把通用 Skill 改造成自己的工作流

大多数时候，你不需要从零写一个 Skill。更省力的做法，是先找一个方向接近的现成版本，再把它改到贴合自己的真实场景。好用的 Skill 往往带着明显的使用痕迹：常用工具是什么、输出格

式怎么定、哪些地方必须确认、哪些动作可以直接做，都写得很具体。

个人 Skill 与团队 Skill 的差异



改造时具体改哪里

- **description** — 加上你自己或团队常用的说法、关键词、触发场景。
- **触发条件和反例** — 确认不会和其他 Skill 冲突。
- **输入规范** — 加上团队特有的来源（飞书文档、内部系统链接）。
- **workflow** — 加上团队必须的检查点（如安全审查、术语一致性）。
- **输出格式** — 改成团队模板（标题结构、标签、归档规则）。
- **停止条件** — 加上团队的风险动作边界（如不自动发布、不自动修改权限）。
- **示例** — 换成团队真实 case。

改造示例：从通用文档 Skill 到团队知识库 Skill

还是用“飞书草稿整理成知识库文章”这个例子。假设你找到一个通用文档润色 Skill，它能改文字，但不知道团队知识库的发布口径；这时不要重写一套，而是先把它改到刚好能覆盖你的真实流程。



这样改完以后，Skill 仍然很小，但已经带上了团队自己的工作痕迹：它知道什么时候能直接整理，什么时候必须停下来等人确认。

把一次完整引导过程沉淀成 Skill

还有一种更稳的写法：先别急着抽象，完整带 Agent 做一次任务。等这条路真的跑通了，再把过程中反复纠正过的地方整理成 Skill。这样写出来的内容通常更贴近实际，因为它来自一次真实协作，而不是坐在原地想象流程。

这时可以使用 [skill-creator](#)。它适合把一次已经跑通的工作流整理成可复用的 Skill：先理解具体使用样例，再规划 [SKILL.md](#)、[references/](#)、[scripts/](#)、[examples/](#) 这些内容，最后做校验和迭代。

一个 Skill 通常不是一次写成的，而是在聊天里磨合出来的。你先带 Agent 做一遍，指出它哪里理解错、哪里漏步骤、哪里输出不合格；再让强模型把这些修正沉淀成 [SKILL.md](#)、示例和检查清单；最后用下一次真实任务验证它是否真的减少了解释和返工。

复盘时重点看什么



沉淀流程

完整的沉淀路径如下。重点不是“写一份很漂亮的说明”，而是把一次协作里反复纠正过的判断，变成下一次可复用的约束。

1. 先在聊天里完成一次真实任务，不急着抽象，让 Agent 暴露理解偏差和步骤遗漏。
2. 人持续纠偏：哪些地方必须确认、哪些动作不能自动做、哪些输出不算合格，都要在对话中说清楚。
3. 让强模型复盘整段聊天，提炼触发条件、输入要求、固定步骤、失败处理和验收标准。
4. 沉淀 `SKILL.md`，把长资料放进 `references/`，把稳定动作做成 `scripts/` 或 CLI 示例。
5. 换一个同类任务，让较弱模型按 Skill 执行，看它是否还能完成，不再依赖临场解释。
6. 根据失败点继续改 Skill：补触发边界、补反例、补检查清单，直到它在真实任务里稳定减少返工。

建议：先用真实任务跑出一条完整路径，再让强模型把这条路径整理成 Skill。随后用更弱、更便宜的模型执行同类任务来验收它。如果弱模型也能按流程产出稳定结果，说明 Skill 真的把经验写进了流程；如果跑偏，说明人的约束还不够明确，需要继续补边界、补示例、补失败处理。

如果没有合适的现成 Skill，或者场景强依赖内部系统，那就把这次完整引导过程当作第一版素材。写 Skill 不一定从空白页开始，也可以从一次成功的协作开始。

```
改造后的 description
---
name: team-kb-writer
description: 当团队成员提供飞书草稿、会议纪要或技术方案，并要求整理成可发布的团队知识库文章时使用。先读
---
```


改造后的 workflow 可能增加以下步骤：

1. 检查来源文档权限，确认 Agent 可读。
2. 读取原文目录，判断篇幅和章节结构。
3. 按团队模板重组：标题 → 导语 → 正文章节 → 参考资料。
4. 标注待确认事实和缺失来源。
5. 输出到指定知识库空间（不自动发布）。

判断改造是否有效，不看它写得多么完整，而看下一次真实任务里，你是不是少解释了几句，少改了几轮，Agent 有没有在该停的时候停下来。



从零写一个 Skill，最小版本该长什么样？



图 9：从零写 Skill 的最小结构

第一版不要追求完整。先把触发、流程、输出、停止条件写清楚。能跑通一个小场景，比覆盖十个模糊场景更有价值。

先写 description

description 是 Skill 里最容易被低估的一行。它不是写给人看的功能介绍，而是写给 Agent 的路由触发器：用户怎么说、提供了什么输入、期待什么结果时，Agent 应该加载这个 Skill。正

文里的 workflow 可以详细，但 description 要克制；它只负责让 Agent 在正确的时机想起这个 Skill，而不是提前讲完整流程。

写 description 时可以先问三个问题：用户真实会怎么提出这个需求？哪些相邻需求不应该触发？加载这个 Skill 后，Agent 应该产出什么类型的结果？如果这三个问题答不清，先不要急着写正文。



再写 SKILL.md 骨架

最小可用骨架

name: doc-polisher

description: 当用户提供飞书文档、Markdown 草稿或主题大纲，并要求整理成可发布的中文知识库文章时使用。

文档润色 Skill

触发条件

- 用户要求润色、改写、去 AI 味、整理成知识库文章。
- 用户提供草稿、链接或明确主题。

不适用

- 不处理法律、财务、医疗等需要专业审定的最终意见。
- 不凭空补事实；缺资料时先标注假设或追问。

工作流程

1. 读取原文或素材。
2. 提炼目标读者、用途和主线。
3. 先重组结构，再改句子。
4. 删除空泛表达、过度口号和模板化结尾。
5. 输出可直接发布的版本，并列需要人工确认的事实。

输出要求

- 标题明确。
- 每节只讲一个问题。
- 结尾给读者下一步能做的动作。

别只写原则，要写动作



把确定动作交给脚本或 CLI

校验 JSON、统计字数、抽取标题、读取飞书文档、更新文档、跑测试——这些动作不要每次让模型重想。能脚本化就脚本化，能用 CLI 就把命令和参数写清楚。

为什么要脚本化

模型擅长理解意图和生成内容，但不擅长精确记住每个工具的参数组合。如果一个动作每次都一样（读哪个文件、用什么参数、怎么处理失败），把它写成脚本或 CLI 示例，比让模型每次“想起来”稳定得多。

哪些动作适合脚本化

- 每次都执行且参数固定的动作（如读取飞书文档目录、校验 JSON 格式）。
- 涉及多步组合的操作（如先 fetch → 判断结构 → 按 section 精读）。
- 失败后有固定重试或回退路径的动作。
- 需要精确参数、人容易记错的命令。

怎么在 Skill 里写 CLI 示例

不要只写“读取文档内容”。更好的写法是直接给出命令、参数选择逻辑和失败处理：

文档 Skill 中的 CLI 示例

第一步：读取文档目录，判断结构

```
lark-cli docs +fetch --api-version v2 --doc "$DOC_URL" --scope outline --max-depth 3
```

第二步：按章节精读目标段落（不要全量 fetch）

```
lark-cli docs +fetch --api-version v2 --doc "$DOC_URL" --scope section --start-block-id "$HEADID"
```

第三步：局部更新

```
lark-cli docs +update --api-version v2 --doc "$DOC_URL" --command block_replace --block-id "$BLOCKID"
```

权限失败时：提示用户执行授权

```
# lark-cli auth login --scope "docx:document:readonly"
```

脚本化 vs 写进 workflow 的判断标准



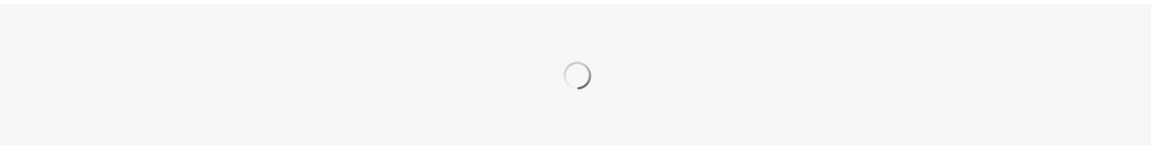
脚本化不是越多越好。只有当一个动作反复出现、参数稳定、且人工记忆容易出错时，才值得抽成脚本。过早脚本化会增加维护成本。

验收清单

写完第一版 Skill 后，用这个清单检查它是否能投入使用：

- 用户换一种常见说法时，description 仍能触发。
- 缺少必要输入时，Skill 知道该追问、假设还是停止。
- 每个步骤都有明确动作，而不只是原则性描述。
- 输出格式固定，能被人或下一个流程继续使用。
- 至少有 3 个标准样例、2 个边界样例、1 个反例。
- 关键 CLI 或脚本调用有参数示例和失败处理。
- 停止条件明确：缺资料、权限失败、高风险动作、连续失败时有处理路径。

写到这里，Skill 只能算"能跑"。要变成生产力，还要能验证、能修、能退役。



Skill 写完之后，怎么验证、迭代和退役？

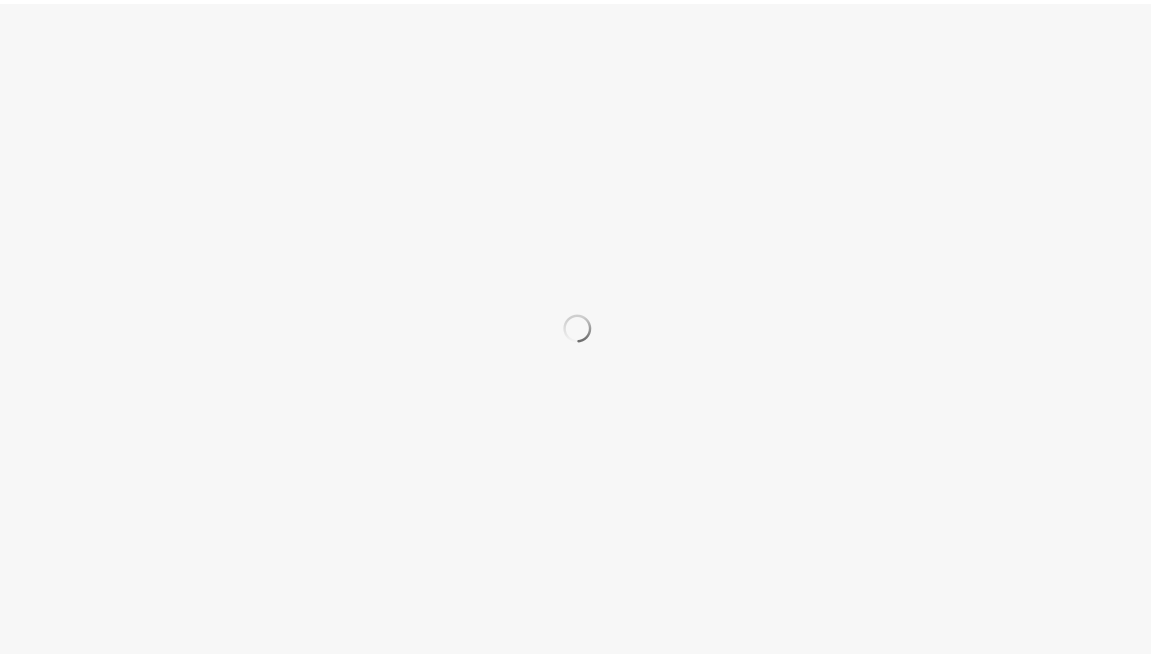
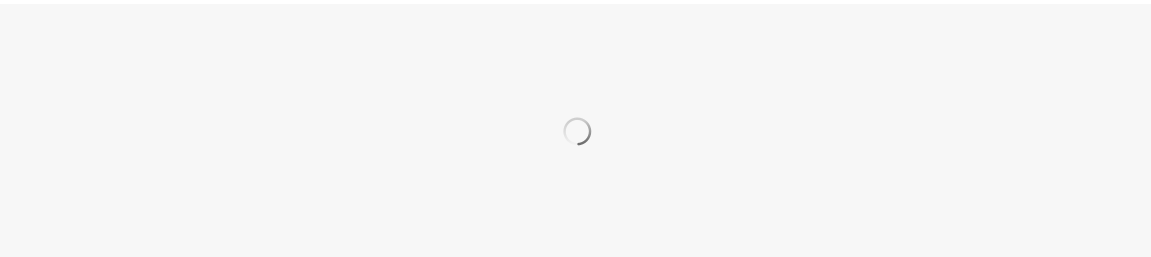


图 10：Skill 的验证、迭代和泛化/特化判断

如果一个 Skill 只能靠"我感觉这次不错"来判断质量，它还没进入生产力阶段。真正能跑起来的 Skill，通常都有评测、版本和复盘。

用 Evals 抓回归

Evals 可以理解为：给一组输入，让 Skill 产出结果，再按标准判断结果好不好。它不是为了证明作者写得对，而是为了快速发现回归。



样本建议：核心样本 5-10 个（用户最常提交的任务），边界样本 3-5 个（空输入、资料不全、格式混乱），已知坑 3-5 个（之前误触发、乱用工具的案例）。

还是沿用前面的飞书草稿案例。一个最小的 eval 文件可以先长这样：不用把评测系统做得很复杂，但要把真实任务、期望结果和断言写清楚。这样每次改 Skill 后，才能用同一把尺子回看它到底有没有变好。

evals.json 示例

```
{
  "skill_name": "team-kb-writer",
  "evals": [
    {
      "id": 1,
      "prompt": "把 workspace/kb-drafts/product-launch-notes.md 整理成一篇可发布的团队知识库文章。",
      "expected_output": "产出一篇结构化知识库文章，包含标题、导语、章节结构、callout、来源链接、待确认事项",
      "assertions": [
        { "id": "a1", "text": "识别出目标读者和发布目标" },
        { "id": "a2", "text": "输出包含标题、导语和清晰的章节结构" },
        { "id": "a3", "text": "保留原始来源链接，没有凭空补事实" },
        { "id": "a4", "text": "列出待确认事实和发布前检查项" },
        { "id": "a5", "text": "没有自动发布、修改权限或 @ 人" }
      ],
      "files": []
    },
    {
      "id": 2,
      "prompt": "workspace/kb-drafts/messy-meeting-notes.md 是一份结构混乱的会议纪要，请整理成团队知识库文章。",
      "expected_output": "将混乱纪要重组为可阅读的知识库文章，并把无来源数据、口径不清的结论和需要人工确认事项单独列出",
      "assertions": [
        { "id": "a1", "text": "没有照搬原始会议纪要顺序，而是重组为文章结构" },
        { "id": "a2", "text": "识别出缺少来源的数据或结论" },
        { "id": "a3", "text": "把待确认事实单独列出" },
        { "id": "a4", "text": "没有把不确定内容写成确定结论" }
      ],
      "files": []
    },
    {
      "id": 3,
      "prompt": "我改了 team-kb-writer 的 SKILL.md，把待确认事实的规则写得更严格了。请在 team-kb-writer 目录下重跑评测，生成新的 grading 和 benchmark，并对比 iteration-1 和 iteration-2 的结果。",
      "expected_output": "在 iteration-2 目录重跑评测，生成新的 grading 和 benchmark，并对比 iteration-1 和 iteration-2 的结果",
      "assertions": [
        { "id": "a1", "text": "创建 iteration-2 目录，而不是覆盖 iteration-1" },
        { "id": "a2", "text": "重新运行评测，非复用 iteration-1 数据" },
        { "id": "a3", "text": "生成 iteration-2 的 benchmark 报告" },
        { "id": "a4", "text": "对比 iteration-1 和 iteration-2 的通过率或失败点" },
        { "id": "a5", "text": "说明规则变严后是否出现误伤" }
      ],
      "files": []
    }
  ]
}
```

```
]
}
```

这个结构里，`prompt` 是真实任务，`expected_output` 是希望 Agent 最终交付什么，`assertions` 则把“做得对”拆成可检查的细项。第一类 case 检查标准草稿能不能被稳定整理；第二类 case 用结构混乱、事实来源缺失的输入来测边界；第三类 case 用来比较新版 Skill 和旧版 Skill，避免改了一条规则却引入新的问题。

失败分类与处理

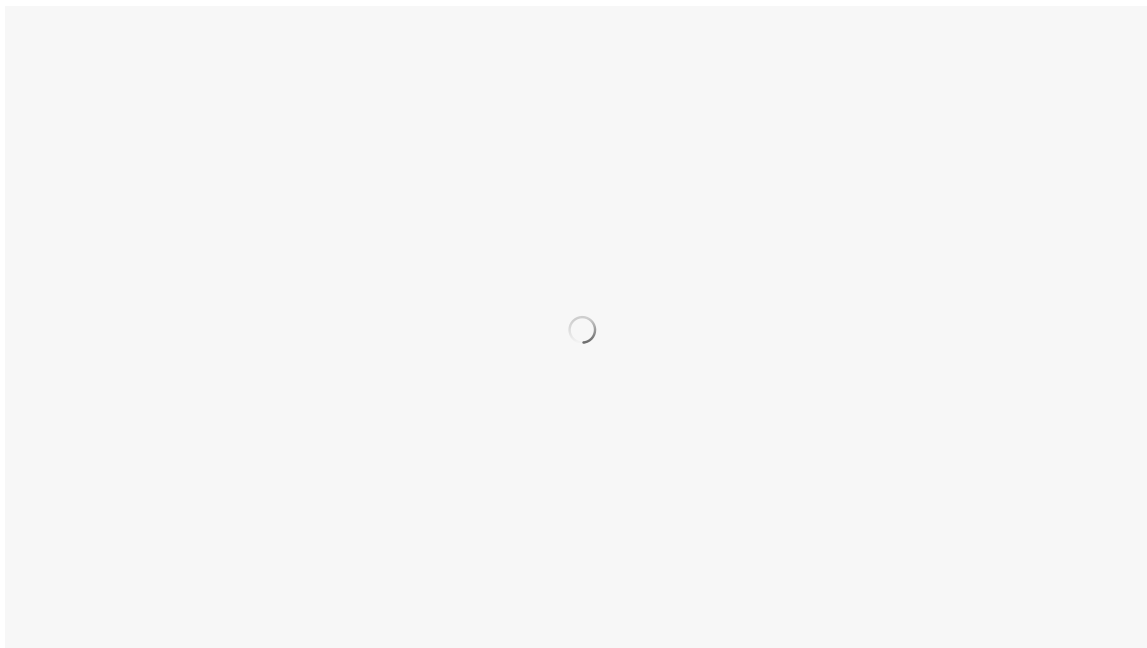
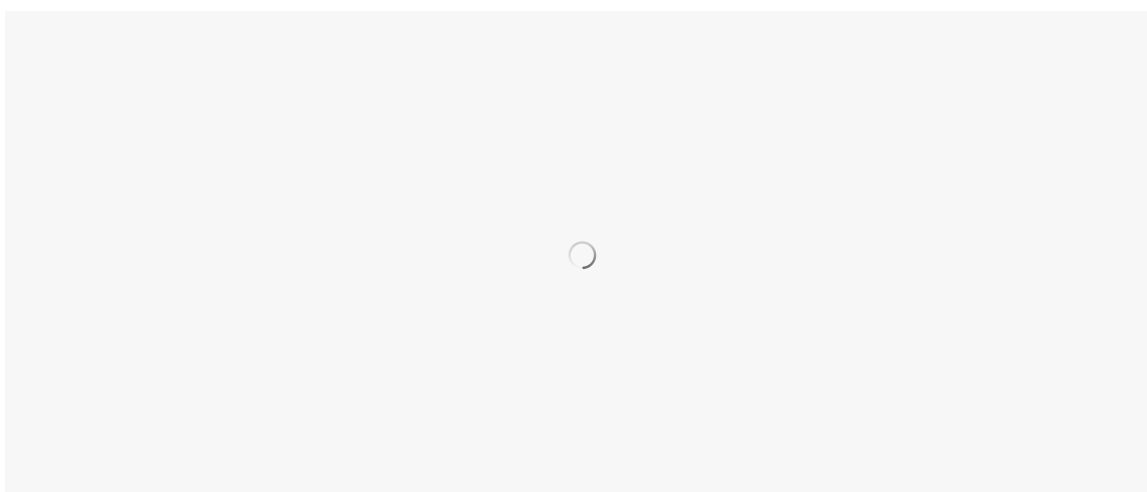


图 11：失败类型与修复动作

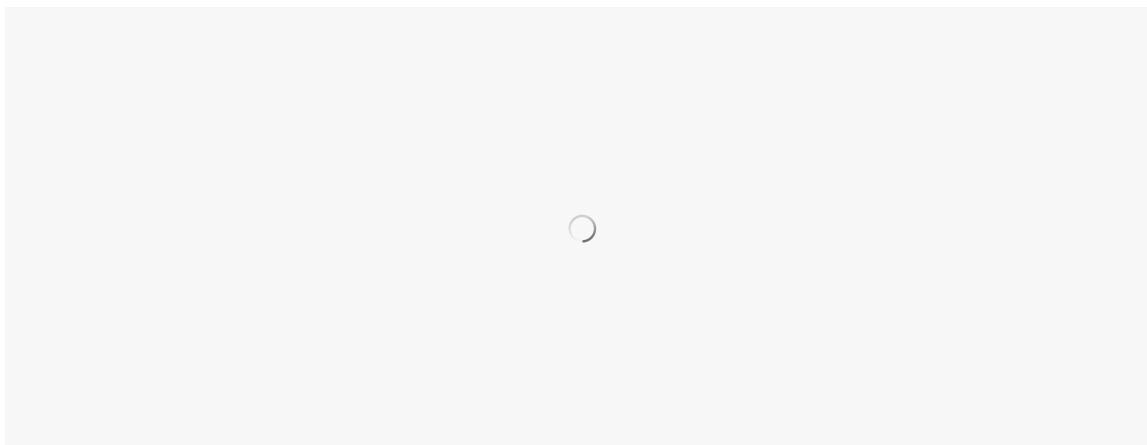


持续迭代：从日常 case 驱动改进

一个 Skill 的成熟过程不是“写完 → 发布 → 结束”，而是一个小循环：遇到 case → 跑 Skill → 记录问题 → 改 description / workflow / examples / scripts → 用下一个 case 验证。

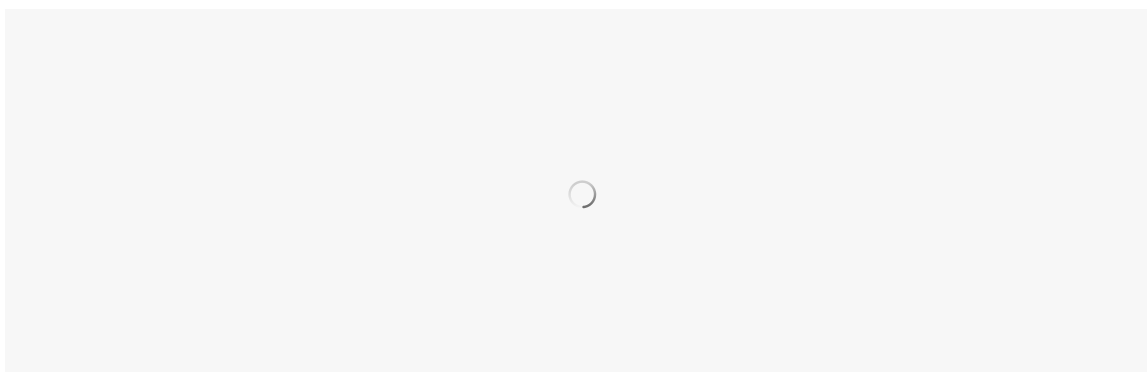
迭代循环

每次失败都是改进的素材：



Gotchas 飞轮：把真实失败变成边界

维护 Skill 时，不要每次遇到问题就重写整份说明。更常见、也更有效的做法，是把真实失败追加成 gotcha：什么情况下不要触发、哪个工具容易误用、哪类输入看起来相似但其实不该走这条流程、连续失败时应该停在哪里。



gotchas 的价值在于它来自真实任务，而不是想象中的完美流程。一个 Skill 越成熟，它不一定越长，但会更清楚地知道哪些路不能走。

让 AI 辅助你创建和改进 Skill

你可以把 AI 当成“流程整理搭子”。先把一次完整协作过程交给 AI，让它帮你复盘：哪些步骤是可复用的，哪些判断是场景相关的，哪些资料应该放进 references，哪些动作值得脚本化。



评测建议

评测不要只看“有 Skill 的结果”，最好有 baseline。新建 Skill 时，可以比较“有 Skill vs 无 Skill”；优化 Skill 时，可以比较“新版 vs 旧版”。这样你才能知道这次修改到底让结果变好了，还是只是看起来更复杂了。

改 description 要同时补正例和反例

description 决定 Skill 是否会被加载，所以它不是普通文案。把它写宽一点，可能会误触发；写窄一点，可能会漏触发。更麻烦的是，新增一个 Skill 也可能影响已有 Skill：两个 Skill 的触发范围如果重叠，Agent 就会在相似任务里摇摆，最后表现得像“不稳定”。

只要改 description，最好同步补两类评测：一类是必须触发的正例，另一类是看起来相似但禁止触发的反例。这样才能判断这次修改是真的改善了路由，还是只是让 Skill 看起来更完整。

如果一个 Skill 经常靠扩 description 才能被想起，通常说明它的边界还没有想清楚。此时不妨回到真实 case：用户到底怎么提需求？哪些说法稳定出现？哪些相邻任务应该交给别的 Skill 或普通对话处理？

什么时候泛化，什么时候特化



建议：从特化开始，用真实 case 验证；当同一类规则在多个 case 里反复出现，再把它泛化。

什么时候该退役

不是所有 Skill 都应该永远存在。以下情况可以考虑合并、降级或删除：

- 模型原生能力已经覆盖该场景。
- 维护成本超过收益。
- 流程已经过时。
- 和新 Skill 大量重叠。

退役不是失败。Skill 太多而且互相打架，Agent 反而更难做对。留下能被复用、能被测试、能被别人接手维护的那部分就好。



今天开始，怎么把一个重复任务变成 Skill？

开始时不用想得太大。选一个真实场景，先找现成 Skill，能直接用就试跑，差一点就改；如果确实找不到合适的，再写一个最小版。跑起来以后，失败点也别放过，它们正是下一版规则、样例和评测的来源。

Skill 的价值不在于让 Agent 显得更聪明，而在于把那些总被重复解释的做法留下来。你踩过的坑、改过的口径、确认过的边界，如果每次都靠聊天临时补，迟早还会再漏一次。

个人 Skill 记录的是自己的工作习惯，团队 Skill 记录的是协作里的隐性规则。一个 Skill 如果可以被复用、被测试、被别人接手维护，就已经从个人经验变成了团队以后可以继续使用的流程。

今天就能开始的 6 个动作



图 12：今天就能开始的 6 个动作

- 选一个本周已经重复出现过的 case，最好有真实输入和真实交付物。
- 写一句清楚的 **description**：什么时候触发、输入是什么、输出是什么。
- 列出 5 步以内的 workflow，并标出哪一步不能省。
- 放 1 个好样例和 1 个反例，避免 Agent 只理解原则。
- 明确停止条件：缺资料、权限失败、高风险动作、事实不确定时怎么处理。
- 用同一个真实 case 跑一遍，记录失败点，再改下一版。

第一版 Skill 不需要照顾所有情况。先让一个小任务下次跑得更稳：少问一轮背景，关键步骤不漏，遇到风险知道停下来。做到这一点，就值得留下继续磨。



TRA E.ai 

TRA E, The Real AI Engineer | 字节跳动旗下的AI编程产品, 你的专属AI开发工程师。... >
166篇原创内容

公众号

最佳实践 · 目录 ≡

< 上一篇 · 新手必看: 3 个真实场景, 讲透 SOLO 高效用法

